U24AI006

ASSIGNMENT -10

1. The union government is planning to set up 3 new airports in the district of Surat due to the newly built IT Infrastructure, economic corridors, and diamond hub. You are an official in the Airport Authority of India. You are tasked to find 3 locations that are suitable for the airports. While choosing the locations, you have to keep in mind that the sum of squares of the distances from nearest airports should be minimum for all cities/towns/villages in Surat. The coordinates of the cities are given to you (you will get a file named cities.csv). You need to use the techniques below to complete your task that satisfies the mentioned criteria.
    a. Using First Order Derivatives/Gradients (Gradient Descent Method)
    b. Using Second Order Derivatives/Hessian (Newton Raphson Method)

    Compare the two techniques by finding the sum of squared distances of each uster/airport.

2. You are building a vacuum cleaning agent that can clean the floor tiles. You are given a simple setting of cleaning two tiles A and B where each percept contains two inputs of the location and state of the tile. But the agent goes erratic in the following ways:
    a. When applied to a dirty square the action cleans the square and sometimes cleans up dirt in an adjacent square, too.
    b. When applied to a clean square the action sometimes deposits dirt on the carpet.

    Design this erratic agent with AND-OR graph search algorithm.

CODE(1):

```
import numpy as np
import pandas as pd
import os

script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(script_dir, 'cities.csv')
data = pd.read_csv(csv_path, encoding='latin1', on_bad_lines='skip')
points = data.values

def kmeans_gradient_descent(points, k, max_iter=100, lr=0.01):
    np.random.seed(42)
    centroids = points[np.random.choice(len(points), k, replace=False)].copy()

    for _ in range(max_iter):
        distances = np.sqrt(((points[:, np.newaxis] - centroids) ** 2).sum(axis=2))
        labels = np.argmin(distances, axis=1)

        grad = np.zeros_like(centroids)
        for i in range(k):
            mask = labels == i
            if mask.sum() > 0:
                grad[i] = -2 * (points[mask] - centroids[i]).sum(axis=0)
```

```
        centroids -= lr * grad

    distances = np.sqrt(((points[:, np.newaxis] - centroids) ** 2).sum(axis=2))
    labels = np.argmin(distances, axis=1)

    total_cost = 0
    for i in range(k):
        mask = labels == i
        if mask.sum() > 0:
            total_cost += ((points[mask] - centroids[i]) ** 2).sum()

    return centroids, labels, total_cost

def kmeans_newton_raphson(points, k, max_iter=20):
    np.random.seed(42)
    centroids = points[np.random.choice(len(points), k, replace=False)].copy()

    for _ in range(max_iter):
        distances = np.sqrt(((points[:, np.newaxis] - centroids) ** 2).sum(axis=2))
        labels = np.argmin(distances, axis=1)

        for i in range(k):
            mask = labels == i
            if mask.sum() > 0:
                cluster_points = points[mask]
                n = len(cluster_points)
                gradient = -2 * (cluster_points - centroids[i]).sum(axis=0)
                hessian = 2 * n * np.eye(2)

                try:
                    delta = np.linalg.solve(hessian, gradient)
                    centroids[i] -= delta
                except:
                    pass

    distances = np.sqrt(((points[:, np.newaxis] - centroids) ** 2).sum(axis=2))
    labels = np.argmin(distances, axis=1)

    total_cost = 0
    for i in range(k):
        mask = labels == i
        if mask.sum() > 0:
            total_cost += ((points[mask] - centroids[i]) ** 2).sum()

    return centroids, labels, total_cost
```

```
k = 3

centroids_gd, labels_gd, cost_gd = kmeans_gradient_descent(points, k)
centroids_nr, labels_nr, cost_nr = kmeans_newton_raphson(points, k)

print("Gradient Descent Method:")
print("Centroids:")
for i, c in enumerate(centroids_gd):
    print(f"  Airport {i+1}: ({c[0]:.3f}, {c[1]:.3f})")
print(f"Sum of squared distances: {cost_gd:.3f}")
print()

print("Newton-Raphson Method:")
print("Centroids:")
for i, c in enumerate(centroids_nr):
    print(f"  Airport {i+1}: ({c[0]:.3f}, {c[1]:.3f})")
print(f"Sum of squared distances: {cost_nr:.3f}")
print()

print("Comparison:")
print(f"Gradient Descent cost: {cost_gd:.3f}")
print(f"Newton-Raphson cost: {cost_nr:.3f}")
if cost_gd < cost_nr:
    print("Gradient Descent is better")
elif cost_nr < cost_gd:
    print("Newton-Raphson is better")
else:
    print("Both are equal")
```

CODE(2):

```
def and_or_search(problem):
    return or_search(problem, problem['initial'], [])

def or_search(problem, state, path):
    if problem['is_goal'](state):
        return []
    if state in path:
        return 'failure'
    for action in problem['actions'](state):
        plan = and_search(problem, problem['results'](state, action), [state] +
path)
        if plan != 'failure':
            return [action] + plan
    return 'failure'

def and_search(problem, states, path):
    plans = []
    for s in states:
        plan = or_search(problem, s, path)
        if plan == 'failure':
            return 'failure'
        plans.append((s, plan))
    return plans

def is_goal(state):
    return state['A'] == 'clean' and state['B'] == 'clean'

def actions(state):
    if state['loc'] == 'A':
        return ['clean', 'right']
    else:
        return ['clean', 'left']

def results(state, action):
    loc = state['loc']
    A = state['A']
    B = state['B']

    if action == 'clean':
        if loc == 'A':
            if A == 'dirty':
                return [
                    {'loc': 'A', 'A': 'clean', 'B': B},
                    {'loc': 'A', 'A': 'clean', 'B': 'clean'}
```

```
                ]
            else:
                return [
                    {'loc': 'A', 'A': 'clean', 'B': B},
                    {'loc': 'A', 'A': 'dirty', 'B': B}
                ]
        else:
            if B == 'dirty':
                return [
                    {'loc': 'B', 'A': A, 'B': 'clean'},
                    {'loc': 'B', 'A': 'clean', 'B': 'clean'}
                ]
            else:
                return [
                    {'loc': 'B', 'A': A, 'B': 'clean'},
                    {'loc': 'B', 'A': A, 'B': 'dirty'}
                ]
    elif action == 'right':
        return [{'loc': 'B', 'A': A, 'B': B}]
    elif action == 'left':
        return [{'loc': 'A', 'A': A, 'B': B}]

def format_state(s):
    return f"[Loc={s['loc']}, A={s['A']}, B={s['B']}]"

def print_plan(plan, indent=0):
    prefix = "  " * indent
    i = 0
    while i < len(plan):
        item = plan[i]
        if isinstance(item, str):
            print(f"{prefix}Action: {item}")
            branches = []
            while i + 1 < len(plan) and isinstance(plan[i + 1], tuple):
                branches.append(plan[i + 1])
                i += 1
            if branches:
                for state, subplan in branches:
                    print(f"{prefix}  IF outcome {format_state(state)}:")
                    if subplan:
                        print_plan(subplan, indent + 2)
                    else:
                        print(f"{prefix}    -> Goal reached")
        i += 1
```

U24AI006

```python
initial_state = {'loc': 'A', 'A': 'dirty', 'B': 'dirty'}

problem = {
    'initial': initial_state,
    'is_goal': is_goal,
    'actions': actions,
    'results': results
}

print(f"Initial State: {format_state(initial_state)}")
print(f"Goal: Both tiles A and B should be clean")
print()

plan = and_or_search(problem)

if plan == 'failure':
    print("No conditional plan found.")
else:
    print("Conditional Plan:")
    print("-" * 30)
    print_plan(plan)
```

OUTPUT:

```
Initial State: [Loc=A, A=dirty, B=dirty]
Goal: Both tiles A and B should be clean

Conditional Plan:
------------------------------
Action: clean
  IF outcome [Loc=A, A=clean, B=dirty]:
    Action: right
      IF outcome [Loc=B, A=clean, B=dirty]:
        Action: clean
          IF outcome [Loc=B, A=clean, B=clean]:
            -> Goal reached
          IF outcome [Loc=B, A=clean, B=clean]:
            -> Goal reached
  IF outcome [Loc=A, A=clean, B=clean]:
    -> Goal reached
```